

AI Assisted Coding

Assignment 9.3

Name: P. Vineeth Kumar

Hall ticket no: 2303A51256

Batch no: 19

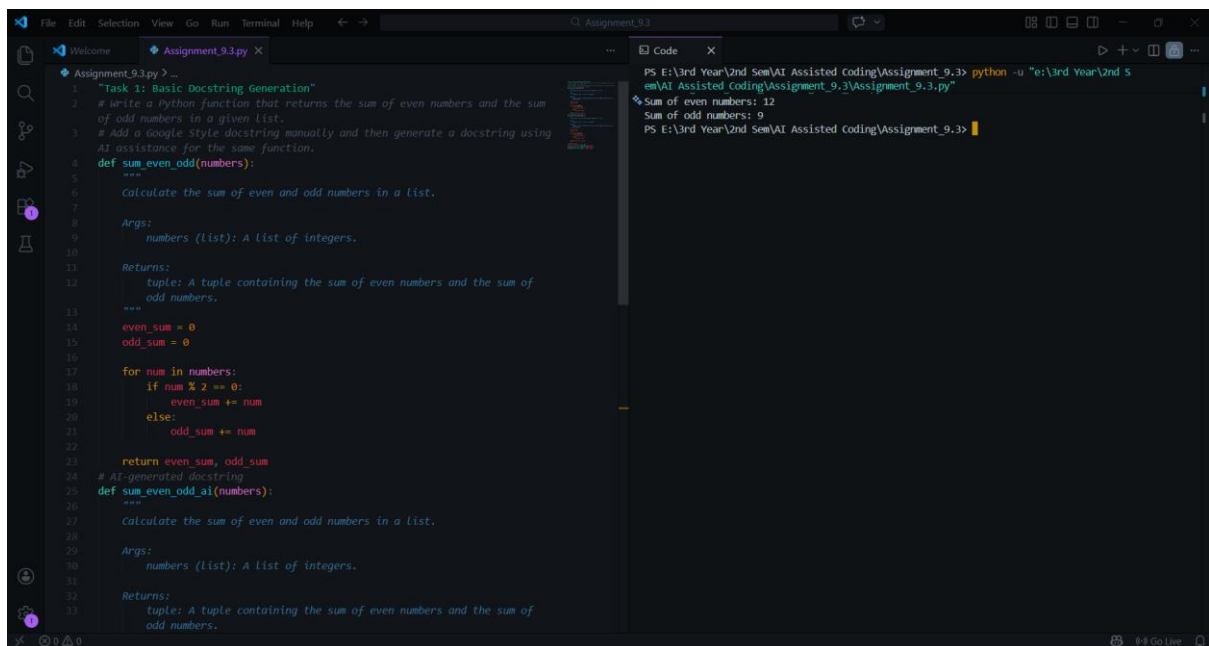
Task 1: Basic Docstring Generation

Prompt:

Write a Python function that returns the sum of even numbers and the sum of odd numbers in a given list.

Add a Google Style docstring manually and then generate a docstring using AI assistance for the same function.

Code & Output:



```
File Edit Selection View Go Run Terminal Help
Assignment_9.3.py X
Task 1: Basic Docstring Generation
1 # Write a Python function that returns the sum of even numbers and the sum
2 # of odd numbers in a given list.
3 # Add a Google Style docstring manually and then generate a docstring using
4 # AI assistance for the same function.
5 def sum_even_odd(numbers):
6     """
7     Calculate the sum of even and odd numbers in a list.
8
9     Args:
10         numbers (list): A list of integers.
11
12     Returns:
13         tuple: A tuple containing the sum of even numbers and the sum of
14             odd numbers.
15     """
16     even_sum = 0
17     odd_sum = 0
18     for num in numbers:
19         if num % 2 == 0:
20             even_sum += num
21         else:
22             odd_sum += num
23     return even_sum, odd_sum
24 # AI-generated docstring
25 def sum_even_odd_ai(numbers):
26     """
27     Calculate the sum of even and odd numbers in a list.
28
29     Args:
30         numbers (list): A list of integers.
31
32     Returns:
33         tuple: A tuple containing the sum of even numbers and the sum of
34             odd numbers.
35     """
36     even_sum = 0
37     odd_sum = 0
38     for num in numbers:
39         if num % 2 == 0:
40             even_sum += num
41         else:
42             odd_sum += num
43     return even_sum, odd_sum
44
45 # Test the function
46 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
47 result = sum_even_odd(numbers)
48 print(f"Sum of even numbers: {result[0]}")
49 print(f"Sum of odd numbers: {result[1]}")
50
51 PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_9.3> python -u "e:\3rd Year\2nd S
52 em\AI Assisted Coding\Assignment_9.3\Assignment_9.3.py"
53 Sum of even numbers: 12
54 Sum of odd numbers: 9
55 PS E:\3rd Year\2nd Sem\AI Assisted coding\Assignment_9.3>
```

```
def sum_even_odd(numbers):  
    return even_sum, odd_sum  
    # AI-generated docstring  
def sum_even_odd_ai(numbers):  
    """  
    Calculate the sum of even and odd numbers in a list.  
  
    Args:  
        numbers (list): A list of integers.  
  
    Returns:  
        tuple: A tuple containing the sum of even numbers and the sum of  
        odd numbers.  
    """  
    even_sum = 0  
    odd_sum = 0  
    for num in numbers:  
        if num % 2 == 0:  
            even_sum += num  
        else:  
            odd_sum += num  
    return even_sum, odd_sum  
  
# Example usage  
numbers = [1, 2, 3, 4, 5, 6]  
even_sum, odd_sum = sum_even_odd(numbers)  
print(f'Sum of even numbers: {even_sum}')  
print(f'Sum of odd numbers: {odd_sum}')
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_9.3> python -u "e:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_9.3\Assignment_9.3.py"  
Sum of even numbers: 12  
Sum of odd numbers: 9  
PS E:\3rd Year\2nd Sem\AI Assisted coding\Assignment_9.3>
```

Explanation:

The manual docstring provides a more descriptive explanation of the function's purpose and return structure. It clearly states the argument type and explains the output in sentence form. The AI-generated docstring is shorter and more concise but still correct. However, it lacks deeper descriptive clarity compared to the manual version. This comparison highlights that AI-generated documentation is accurate but sometimes less detailed, requiring human refinement for completeness.

Task 2: Automatic Inline Comments

Prompt:

Generate a Python class named `sru_student` with attributes `name`, `roll_no`, and `hostel_status`, and methods `fee_update()` and `display_details()`. Add inline comments automatically.

Code & Output:

```
51
52 "Task 2: Automatic Inline Comments"
53 # Generate a Python class named sru_student with
  attributes name, roll_no, and hostel_status, and methods
  fee_update() and display_details(). Add inline comments
  automatically.
54 class sru_student:
55     def __init__(self, name, roll_no, hostel_status):
56         # Initialize the student's name, roll number, and
           hostel status
57         self.name = name
58         self.roll_no = roll_no
59         self.hostel_status = hostel_status
60
61     def fee_update(self, new_fee):
62         # Update the fee for the student
63         self.fee = new_fee
64
65     def display_details(self):
66         # Display the student's details
67         print(f"Name: {self.name}")
68         print(f"Roll No: {self.roll_no}")
69         print(f"Hostel Status: {self.hostel_status}")
70 # Example usage
71 student1 = sru_student("Alice", "SRU123", "Active")
72 student1.fee_update(5000)
73 student1.display_details()
74
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_9.3> python -u "e:\3rd Year\2nd Sem\AI Assist
ed Coding\Assignment_9.3\Assignment_9.3.py"
Name: Alice
Roll No: SRU123
Hostel Status: Active
PS E:\3rd Year\2nd Sem\AI Assisted coding\Assignment_9.3>
```

Explanation:

Manual comments are precise and written with developer intent. AI-generated comments are also correct but slightly generic. AI explains obvious lines but may miss contextual explanations. This shows that AI can accelerate documentation but human review is necessary to ensure meaningful and non-redundant comments.

Task 3: Module-Level and Function-Level Documentation

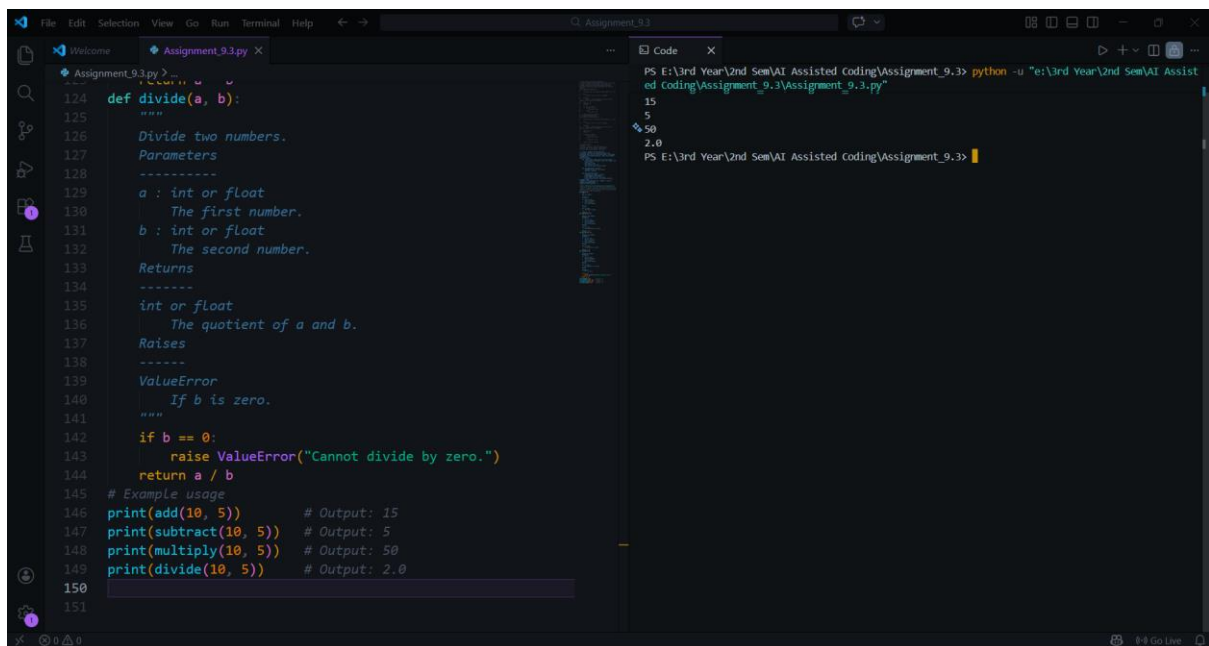
Prompt:

Generate a Python calculator module with functions add, subtract, multiply, and divide. Add NumPy-style docstrings manually and then generate module-level and function-level documentation using AI assistance.

Code & Output:

```
File Edit Selection View Go Run Terminal Help Assignment_9.3
Assignment_9.3.py X
Assignment_9.3.py > ...
74
75 "Task 3: Module-Level and Function-Level Documentation"
76 # Generate a Python calculator module with functions add,
77 subtract, multiply, and divide. Add NumPy-style docstrings
78 manually and then generate module-level and function-level
79 documentation using AI assistance.
80
81 def add(a, b):
82     """
83     Add two numbers.
84
85     Parameters
86     -----
87     a : int or float
88         The first number.
89     b : int or float
90         The second number.
91
92     Returns
93     -----
94     int or float
95         The sum of a and b.
96     """
97     return a + b
98
99 def subtract(a, b):
100     """
101     Subtract two numbers.
102     Parameters
103     -----
104     a : int or float
105         The first number.
106     b : int or float
107         The second number.
108
109     Returns
110     -----
111     int or float
112         The difference of a and b.
113     """
114     return a - b
115
116 def multiply(a, b):
117     """
118     Multiply two numbers.
119     Parameters
120     -----
121     a : int or float
122         The first number.
123     b : int or float
124         The second number.
125
126     Returns
127     -----
128     int or float
129         The product of a and b.
130     """
131     return a * b
132
133 def divide(a, b):
134     """
135     Divide two numbers.
136     Parameters
137     -----
138     a : int or float
139         The first number.
140     b : int or float
141         The second number.
142
143     Returns
144     -----
145     float
146         The quotient of a and b.
147     """
148     return a / b
149
150 if __name__ == '__main__':
151     # Test the functions
152     print(add(1, 2))
153     print(subtract(2, 1))
154     print(multiply(2, 2))
155     print(divide(2, 1))
156
157 PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_9.3> python -u "E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_9.3\Assignment_9.3.py"
5
15
50
2.0
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_9.3>
```

```
File Edit Selection View Go Run Terminal Help Assignment_9.3
Assignment_9.3.py X
Assignment_9.3.py > ...
94 def subtract(a, b):
95     """
96     Subtract two numbers.
97     Parameters
98     -----
99     a : int or float
100         The first number.
101     b : int or float
102         The second number.
103
104     Returns
105     -----
106     int or float
107         The difference of a and b.
108     """
109     return a - b
110
111 def multiply(a, b):
112     """
113     Multiply two numbers.
114     Parameters
115     -----
116     a : int or float
117         The first number.
118     b : int or float
119         The second number.
120
121     Returns
122     -----
123     int or float
124         The product of a and b.
125     """
126     return a * b
127
128 def divide(a, b):
129     """
130     Divide two numbers.
131     Parameters
132     -----
133     a : int or float
134         The first number.
135     b : int or float
136         The second number.
137
138     Returns
139     -----
140     float
141         The quotient of a and b.
142     """
143     return a / b
144
145 if __name__ == '__main__':
146     # Test the functions
147     print(add(1, 2))
148     print(subtract(2, 1))
149     print(multiply(2, 2))
150     print(divide(2, 1))
151
152 PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_9.3> python -u "E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_9.3\Assignment_9.3.py"
5
15
50
2.0
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_9.3>
```



```
def divide(a, b):
    """
    Divide two numbers.
    Parameters
    -----
    a : int or float
        The first number.
    b : int or float
        The second number.
    Returns
    -----
    int or float
        The quotient of a and b.
    Raises
    -----
    ValueError
        If b is zero.
    """
    if b == 0:
        raise ValueError("Cannot divide by zero.")
    return a / b

# Example usage
print(add(10, 5))      # Output: 15
print(subtract(10, 5)) # Output: 5
print(multiply(10, 5)) # Output: 50
print(divide(10, 5))   # Output: 2.0
```

```
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_9.3> python -u "e:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_9.3\Assignment_9.3.py"
15
50
2.0
PS E:\3rd Year\2nd Sem\AI Assisted Coding\Assignment_9.3>
```

Explanation:

Manual NumPy-style docstrings follow a structured scientific documentation format with sections for parameters and returns. AI-generated documentation is concise and suitable for module overviews but lacks deep parameter-level detailing. AI performs well in summarization, while manual documentation provides stronger technical clarity.

Final Conclusion:

This lab demonstrated the role of AI in generating documentation and comments automatically. AI-assisted tools significantly reduce documentation effort and improve consistency. However, human review remains essential to ensure clarity, correctness, and contextual relevance in software documentation.